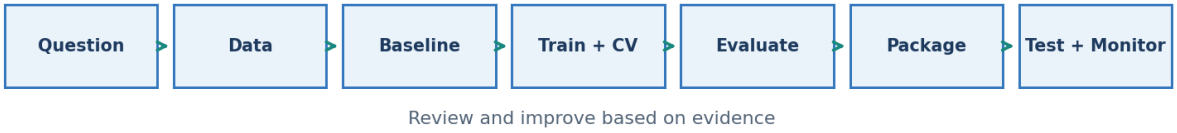
ספר הלימוד המאוחד במדעי הבינה המלאכותית

פרק 14 — ניהול פרויקט במדעי הנתונים

מן המחברת ב־Colab למערכת AI שאפשר לשחזר, לבדוק, להסביר ולהציג

מהדורת תלמיד | כיתות י״א–י״ב | התמחות למידת מכונה ולמידה עמוקה | תכנון לימודי ל־25 שעות



איור 1 — מחזור החיים של פרויקט למידת מכונה: התקדמות עם בדיקות וחזרה לתיקון.

מה נלמד בפרק?

עד כה למדנו כיצד לטעון נתונים, לבצע חקר, לאמן מסווגים ורשתות עצביות ולחשב מדדי הצלחה. אבל במעבדה אמיתית אין די בתא Colab שמפיק Accuracy גבוה. מפתח AI צריך לדאוג שהנתונים יישארו מזוהים, שהקוד ניתן להרצה חוזרת, שהמודל יעבוד גם מחוץ למחברת, ושהחלטות יהיו מתועדות ומבוססות ראיות.

בפרק זה נפתח את עבודתו של דודו: מערכת זיהוי ספרות בכתב יד. לא נבנה רשת עצבית חדשה — כבר למדנו לעשות זאת בפרקים קודמים. הפעם נלמד לעטוף מודל קיים בתהליך הנדסי ומחקרי אחראי. כל שלב ילווה בתוצר שאפשר להציג למורה ולבוחן.

עוגן התכנית וידע קודם

מפת ההתמחות במאגר הקורס מייעדת לפרק 14 — "ניהול פרויקט במדע הנתונים: היבטים מעשיים" — 25 שעות. תתי־הנושאים והמימושים המוצעים כאן הם הרחבה דידקטית מקצועית על סמך ספרי המקור והנחיות הנדסת ML; אין להציג את חלוקת השעות והכלים הספציפיים כציטוט מתוך הסילבוס הרשמי המלא, שעדיין מצריך הצלבה פרטנית.

ידע קודם: Python, NumPy, Pandas, תיעוד Notebook, KNN, חלוקת נתונים, Pipeline, Validation, Test, Accuracy, F1־ו, מטריצת בלבול ועקרונות עיבוד תמונה.

14.1 — סיפור המסגרת: למה קוד שעובד אינו תמיד מוצר?

דודו בנה מחברת שמזהה ספרות באימון. המודל קיבל ציון טוב, ולכן הוא שלח את המחברת לחברה שעובדת במחשב אחר. התברר שהקובץ לא מצא את נתוני האימון, המשתנים נוצרו בסדר הרצה לא ברור, ובמחשב החדש הותקנה גרסת ספרייה אחרת. בשלב הבא ניסו לחזות תמונה חדשה, אבל היא הייתה בגודל 28×28 ולא 8×8. שלוש בעיות שונות נחשפו: ניהול סביבת עבודה, חוזה קלט, ושחזור תוצאות.

הלקח הראשון הוא שיש להבחין בין אב־טיפוס מחקרי לבין מערכת אמינה. אב־טיפוס נועד להוכיח שרעיון עשוי לעבוד. מערכת מסודרת צריכה להגדיר מראש קלט ופלט, לבדוק כשלים, לתעד מגבלות ולאפשר לאדם אחר להפעיל אותה.

14.2 — ניסוח בעיה וקריטריון הצלחה

לפני כתיבת קוד מנסחים מה המערכת מקבלת, מה היא מחזירה, למי היא מיועדת וכיצד נחליט שהצליחה. בדוגמת הספר, הקלט הוא תמונה בגווני אפור של ספרה יחידה בגודל 8×8. הפלט הוא ספרה חזויה מ־0 עד 9. זו משימת סיווג רב־מחלקתי. הנתונים הם תמונות שכבר תויגו, ולכן מדובר בלמידה מונחית.

נגדיר את היעד במחקר: להשוות מסווג פשוט למודל מבוסס מרחק ולבדוק ביצועים על דוגמאות שלא שימשו לבחירת המודל. אחוז הצלחה לבדו אינו מספיק: נדרוש גם Macro F1, ניתוח דוגמאות שגויות וזמן סביר להפקת תחזית. ערכי סף מספריים יתאימו לצורכי הפרויקט ולא ייבחרו לאחר צפייה בתוצאות.

Success criteria = metric + threshold + test protocol + limitations

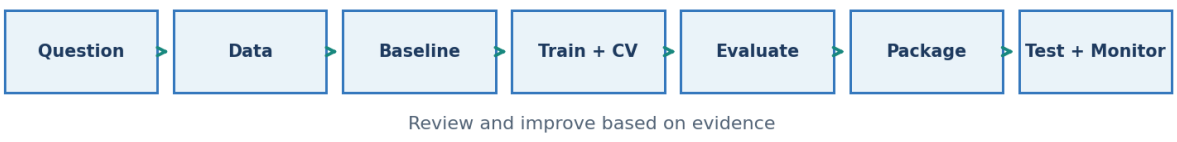
לדוגמה, אפשר להציע מראש כי המודל יעבור מבחן רק אם Macro F1 יהיה לפחות 0.90 על קבוצת Test שמורה, והמערכת תדחה קלט שאינו במבנה המוגדר. זהו קריטריון הוראה מוצע ולא תקן רשמי או התחייבות לביצועים.

14.3 — מסמך דרישות קצר (Project Brief)

רכיב	החלטת הפרויקט	למה צריך להגדירו?
משתמש	תלמיד המעוניין לבדוק ספרה כתובה	מבהיר למי מיועדת המערכת
קלט	64 ערכים מספריים, טווח 0–16	מונע אי־התאמה למאגר
פלט	תווית חזויה 0–9	מונע ערבוב עם הסתברות
איכות	Macro F1 לצד מטריצת בלבול	מונע הסתמכות על מדד יחיד
מגבלות	תמונות 8×8 שדומות למקור	מבהיר אי־התאמה אפשרית לתמונות טלפון
פרטיות	לא משתמשים בתמונות אישיות ללא אישור	ניהול נתונים אחראי

14.4 — שלבי הפרויקט וקריטריוני סיום

אפשר לראות בפרויקט רצף של תוצרים: כרטיס בעיה, כרטיס נתונים, מחברת EDA, Baseline, ניסוי ולידציה, מודל שמור, בדיקות קלט, דוח מסכם והדגמה. לכל שלב נגדיר תנאי סיום. כך התלמיד יודע מתי עבודה הושלמה, והמורה יכול לזהות בעיות לפני שלב מאוחר שבו קשה לתקן אותן.

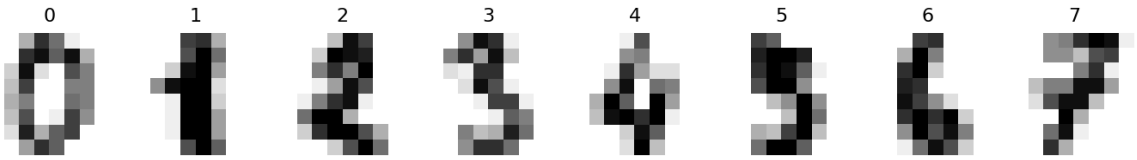


איור 2 — בכל מעבר שלב נדרש תוצר לבדיקה: נתון, קוד, מדד או הסבר.

14.5 — כרטיס נתונים (Data Card)

בכל פרויקט יש לתעד את מקור הנתונים, הרישוי, תאריך האיסוף, משמעות המאפיינים, אופן יצירת התוויות, נתונים חסרים, ייצוג קבוצות אפשרי וסיכונים. כרטיס כזה אינו בירוקרטיה: בלעדיו קשה לדעת אם אפשר להשתמש בנתונים באופן חוקי, האם מותר לפרסם אותם, והאם נתון מסוים יהיה זמין בזמן חיזוי אמיתי.

בדוגמת הספר נשתמש במאגר digits שמגיע עם scikit-learn: 1,797 תמונות, כל תמונה 8×8 פיקסלים בגווני אפור, והערכים במאגר נעים בין 0 ל־16. כל תמונה מיוצגת גם כווקטור באורך 64. המטרה היא ספרה מסומנת. מאגר זה מתאים להדגמת תהליך מבלי לאסוף מידע אישי על תלמידים.



איור 3 — דוגמאות מתוך מאגר *digits* של *scikit-learn*. מספר מעל כל תמונה הוא התווית.

אם נרצה להחליף בהמשך את המאגר בצילומים מהטלפון, אי אפשר להניח שהמודל יעבוד באותה איכות. צילום כזה עשוי להוסיף צבע, צל, סיבוב ורקע, וקיים פער בין תחום נתוני האימון לבין הנתונים החדשים.

14.6 — הפרדת Train, Validation ו־Test

קבוצת Train משמשת ללמידת פרמטרים. קבוצת Validation או קיפולי Cross-Validation משמשים לבחירת מודל והיפר־פרמטרים. קבוצת Test נועדה לאומדן סופי של הכללה אחרי שכל ההחלטות התקבלו. שימוש ב־Test כדי לבחור k ואחר כך דיווח על אותה תוצאה כאילו הייתה עצמאית הוא שגיאה מתודולוגית.



איור 4 — חלוקה מספרית להמחשה בלבד: מתוך 1,000 דוגמאות, 700 לאימון, 150 לוולידציה ו־150 לבדיקה.

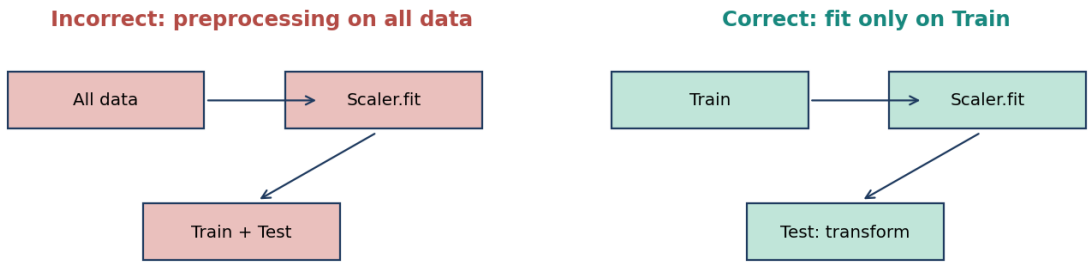
ב־digits נשתמש בדוגמה מודרכת בהפרדה חיצונית של 20% ל־Test, ולאחר מכן נבצע Cross-Validation רק על נתוני Train. נשמור random_state קבוע ונשתמש ב־stratify לשמירה בקירוב על חלקה של כל ספרה בחלוקה. במאגר זמנים או כשיש תמונות רבות מאותו אדם, חלוקה אקראית פשוטה עלולה להטעות; במקרה כזה יש לתכנן חלוקה לפי זמן או לפי קבוצות.

$$1{,}000 \times 0.70 = 700 \;;\; 1{,}000 \times 0.15 = 150 \;;\; 1{,}000 \times 0.15 = 150$$

14.7 — מניעת זליגת מידע כתנאי לאמינות

דוגמה לשגיאה: התלמיד מבצע תקנון על כל המאגר ורק אחר כך מחלק ל־Train ול־Test. כך אפילו ממוצע פשוט חושף לתהליך האימון מידע מנתוני הבדיקה. פתרון נכון: מחלקים קודם, וכל שלב שלומד

נתונים — למשל StandardScaler, בחירת מאפיינים או השלמת חסרים — מותאם ל־Train בלבד. בתוך Cross-Validation, שלבי ההכנה צריכים להילמד מחדש בכל קיפול אימון.



איור 5 — משמאל: פעולה לא נכונה על כל הנתונים; מימין: התאמה לנתוני האימון ושימוש ב־*transform* לנתונים חדשים.

Pipeline מסייע למנוע שגיאה זו משום שהוא כולל באותו אובייקט את שלבי ההכנה והמסווג. פעולה fit מתאימה את השלבים על נתוני האימון, ואילו predict מפעילה על קלט חדש את אותם שלבים בסדר הנכון. Pipeline אינה מתקנת אוטומטית כל דליפה, למשל תווית עתידית שהכנסנו לתכונה.

14.8 — ארגון תיקיות: מחברת אינה כל הפרויקט

כשכל הקוד מצוי במחברת אחת קשה לנהל שינוי, לבדוק רכיב בנפרד או להציג למפתח אחר מה צריך להריץ. לכן נפריד בין נתונים, מחברות חקר, קוד משותף, מודלים שמורים, בדיקות ותיעוד. מבנה מוצע לפרויקט תלמיד:

```
handwritten-digit-project/
    README.md
    requirements.txt
    .gitignore
    data/
        README.md
    notebooks/
        01_exploration.ipynb
        02_experiments.ipynb
    src/
        train.py
        predict.py
    models/
        model_info.json
    reports/
        experiment_log.csv
        model_card.md
    tests/
        test_prediction.py
```

העיקרון חשוב יותר מהשמות: תלמיד אחר צריך לדעת מה הקלטים, מה פותחים קודם, אילו קבצים נדרשים ומהו התוצר. README טוב יכלול את שאלת הפרויקט, המקורות, הוראות התקנה, שיטת הערכה ומגבלות ידועות.

14.9 — ניהול גרסאות עם Git

Git עוקב אחר שינויים בקוד ובתיעוד. כל Commit הוא צילום מצב מוסבר של העבודה. אם ניסוי חדש נשבר, אפשר להבין אילו קבצים השתנו. רצוי לכתוב הודעות Commit בעלות משמעות, כגון "Add train-test split" או "Document prediction input validation", במקום "final final 3".

אין להעלות אוטומטית נתונים אישיים, סיסמאות, מפתחות API וקובצי מודל גדולים ל־Git. ב־gitignore. נציין קבצים זמניים ורגישים. כאשר נדרשת גרסת נתונים גדולה, אפשר ללמוד על כלים כגון DVC כהרחבה — אך אין חובה להתקינם בפרויקט הראשון.

```
# .gitignore - example
__pycache__/
.ipynb_checkpoints/
.env
*.joblib
private_data/
```

14.10 — סביבת עבודה ושחזור (Reproducibility)

ניסוי משוחזר הוא ניסוי שאדם אחר מסוגל להריץ לפי אותן הוראות ולקבל תוצאה עקבית במידה סבירה. כדי לשחזר, יש לשמור את גרסאות הספריות, גרסת המאגר, קוד התהליך, פרמטרים שנבחרו וזרעים אקראיים. random_state מסייע לשלוט בחלק מהאקראיות, אך הוא לא מבטיח שתוצאה תהיה זהה על כל חומרה, גרסת ספרייה או מימוש מקבילי.

ב־requirements.txt מומלץ לתעד גרסאות שתואמות להרצה שנבדקה. אין להעתיק מספרי גרסאות באופן שרירותי; אפשר לייצר רשימת ספריות מתוך הסביבה הפעילה ולסנן אותה לתלויות הנדרשות. רצוי גם לציין גרסת Python ותאריך בדיקה.

14.11 — מודל בסיס (Baseline) לפני סבך של ניסויים

לפני KNN או CNN נבדוק מה משיגה שיטה פשוטה שאינה לומדת קשר בין פיקסלים לספרה, למשל תחזית של הספרה השכיחה ביותר ב־Train. אם יש עשר מחלקות בכמויות דומות, נקבל בערך עשירית מהתחזיות נכונות; התוצאה המדויקת תלויה בחלוקה. בלי Baseline אי אפשר לדעת אם המודל שלנו מוסיף ערך משמעותי.

לפעמים פתרון המבוסס על כללים פשוטים מתאים יותר מפרויקט מסובך. לכן על התלמיד להסביר לא רק איזה מודל טוב יותר, אלא גם מהו מחיר התחזוקה שלו, כיצד נבדק ומהן מגבלותיו.

14.12 — תהליך אימון מודרך ב־Python

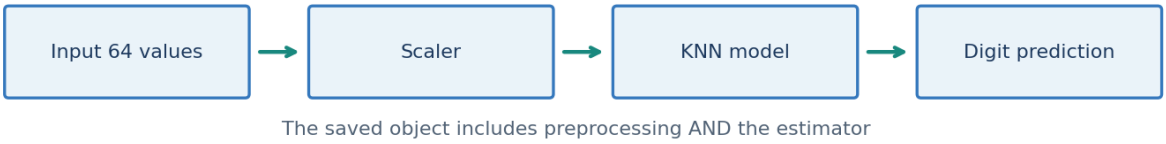
נבצע ניסוי בסיסי על digits. בחרנו להשתמש ב־StandardScaler וב־KNN בתוך Pipeline. הקוד שלהלן ניתן להרצה לפי הסדר; בהמשך הפרק נתרגל הפרדה בין קובץ אימון לקובץ חיזוי. אין לפרש את התוצאה כיכולת לזהות כל תמונה בעולם האמיתי.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score


digits = load_digits()
X, y = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
model = make_pipeline(
    StandardScaler(), KNeighborsClassifier(n_neighbors=3)
)
model.fit(X_train, y_train)
```

```
pred_train = model.predict(X_train)
print('Train Accuracy:', accuracy_score(y_train, pred_train))
print('Train Macro F1:', f1_score(y_train, pred_train, average='macro'))
```

זהו ניסוי הדגמה לקוד, ולא פרוטוקול בחירת מודל סופי: לבחירת מספר השכנים נשתמש ב־Cross-Validation על Train בלבד, ורק אחריה נמדוד ביצועים סופיים על Test שמור.

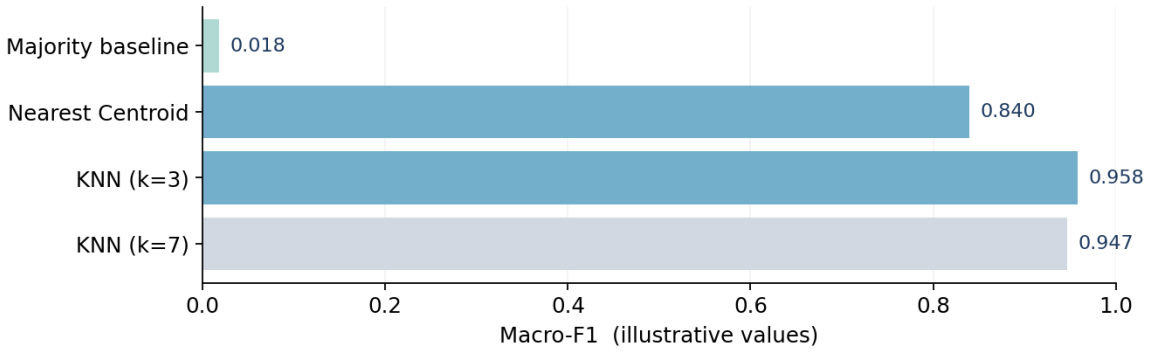


איור 6 — ה־*Pipeline* משמר את אותו עיבוד גם בזמן חיזוי של נתונים חדשים.

14.13 — יומן ניסויים: מה שינינו, ומה למדנו?

ניסוי טוב משנה גורם מתוכנן, מודד תוצאות ומסביר אותן. אם נחליף גם מאפיינים, גם מודל, גם שיטת חלוקה וגם מדד בבת אחת, לא נדע מה גרם לשינוי. לכן נתעד גרסת נתונים, מאפיינים, מודל, היפר־פרמטרים, שיטת ולידציה, מדדים, זמן ריצה ומסקנה.

ניסוי	מה נבדק	מדד CV	החלטה
E01	Baseline של מחלקת הרוב	המחשה — 0.018	נקודת ייחוס
E02	Nearest Centroid	המחשה — 0.840	השוואה ראשונית
E03	KNN עם k=3	המחשה — 0.958	מועמד מוביל
E04	KNN עם k=7	המחשה — 0.947	השוואה נוספת



איור 7 — ערכי הדגמה מומצאים להוראת יומן ניסויים; אינם תוצאות הרצה על *digits*.

בתוך מחברת התלמיד יתועדו תוצאות ההרצה האמיתיות. יש להבדיל בין מספר בדוגמה פדגוגית לבין מספר שהתקבל מריצה, ולשמור את שיטת המדידה כדי שנוכל לפרש פערים.

14.14 — ניתוח תוצאות ולא רק מספר אחד

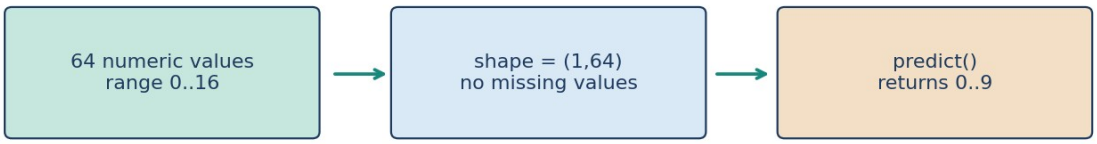
Accuracy שווה למספר התחזיות הנכונות מחולק במספר הבדיקות. אם מתוך 100 דוגמאות 92 סווגו נכון, נקבל 92/100 = 0.92. עם זאת, בסיווג כמה מחלקות חשוב לבדוק גם Macro F1, שמחשב את ממוצע ציוני F1 של המחלקות ללא שקלול לפי גודל המחלקה.

Accuracy = correct predictions / all predictions = 92 / 100 = 0.92

מטריצת בלבול מאפשרת לגלות אילו זוגות ספרות מתבלבלים. אם המודל טועה לעיתים בין 3 ל־8, ייתכן שהתמונות דומות או שמאגר האימון אינו מייצג דיו כתב יד מסוים. הממצא מחייב בדיקה של דוגמאות ולא הסקת סיבה חד־משמעית מתוך המטריצה לבדה.

14.15 — חוזה קלט ופלט (Inference Contract)

כשמעבירים מודל ממחברת למערכת, חובה להגדיר מה מותר להעביר אליו. לדוגמה: מערך של 64 מספרים בטווח 0 עד 16, ללא ערכים חסרים. לאחר הבדיקה נבנה שורה אחת במבנה (1,64) ונפעיל predict. התוצאה היא תווית ספרה ולא הסתברות. אם רוצים להציג הסתברות, נדרש לבדוק מה מחזיר המודל ואת מידת הכיול שלו; אין להפוך מרחק או הצבעת רוב אוטומטית ל"אחוז ביטחון" אמין.



Prediction service contract: validate before calling the model

איור 8 — חוזה הקלט והפלט מונע קריאות שגויות לפני שהן מגיעות למודל.

```
import numpy as np

def predict_digit(values, trained_model):
    arr = np.asarray(values, dtype=float)
    if arr.shape != (64,):
        raise ValueError('Expected exactly 64 values')
    if not np.isfinite(arr).all():
        raise ValueError('All values must be finite')
    if np.any(arr < 0) or np.any(arr > 16):
        raise ValueError('Pixel values must be in 0..16')
    return int(trained_model.predict(arr.reshape(1, -1))[0])
```

הקוד בודק את מבנה קלט הדוגמה, אך מערכת אמיתית זקוקה גם לטיפול מסודר בשגיאות ולהצגת הודעות מובנות למשתמש. שימו לב: תמונת טלפון אינה קלט תואם עד שתעבור תהליך עיבוד מוגדר ומאומת.

14.16 — שמירת מודל ושחזור תחזית

בסיום אימון נשמור את אובייקט ה־Pipeline, כדי שלא נשכח את שלב התקנון בעת הפעלת המודל. אפשר להשתמש ב־joblib עבור קובץ שנוצר על ידינו בסביבה מהימנה. חובה להזהיר כי טעינת קובצי joblib או pickle ממקור לא מהימן עלולה להפעיל קוד; אסור לטעון מודל אקראי שהגיע מאינטרנט או מאדם לא מוכר.

```
from pathlib import Path
import joblib

path = Path('digit_pipeline.joblib')
joblib.dump(model, path)       # save your own trusted model
restored = joblib.load(path)   # only from a trusted source
sample = X_train[0]
assert int(restored.predict([sample])[0]) == int(model.predict([sample])[0])
```

בנוסף למודל נשמור קובץ metadata בפורמט JSON ובו גרסת מודל, שם המאגר, רשימת המאפיינים, גרסאות תוכנה, תאריך ניסוי, שיטת בדיקה ומגבלות. קובץ JSON מכיל תיעוד, לא את המודל עצמו. אין להניח שקובץ מודל ישן יעבוד באופן נתמך אחרי שדרוג ספריות גדול.

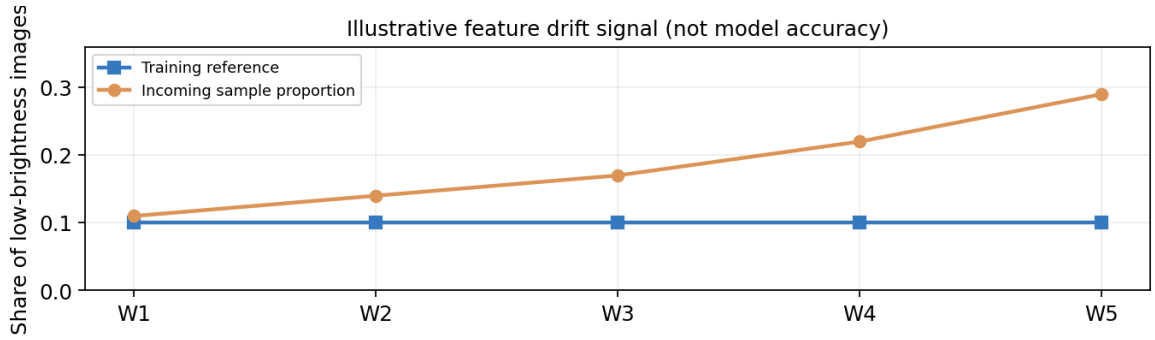
14.17 — בדיקות תוכנה: האם המערכת מתנהגת כמצופה?

בחינת מודל אינה זהה לבדיקת תוכנה. ייתכן שמודל משיג ציון מצוין אך היישום קורס אם משתמש שולח 63 ערכים במקום 64. נבנה בדיקות חיוביות ושליליות: קלט תקין, אורך שגוי, ערך חסר, ערך מחוץ לטווח, קובץ מודל חסר ואי־התאמה בין גרסאות. את הבדיקות נריץ מחדש לאחר שינוי בקוד.

בדיקה	קלט	תוצאה צפויה
חיזוי תקין	64 מספרים בטווח 0–16	ספרה בין 0 ל־9
צורה שגויה	63 מספרים	הודעת שגיאה מובנת
ערך לא חוקי	פיקסל שלילי	דחיית קלט
ערך חסר	NaN	דחיית קלט
שחזור	מודל שמור מהימן	אותה תחזית לדוגמת בדיקה

14.18 — ניטור, שינוי נתונים ותחזוקה

מודל שאומן על כתב יד מסוג אחד עלול לפעול אחרת כשהמשתמשים משתנים. יש להבחין בין שינוי בהתפלגות הקלט (Data Drift) לבין ירידה אמיתית באיכות (Performance Degradation). ניתן לזהות שינוי בסטטיסטיקות של הקלט גם לפני שיש לנו תוויות אמת, אך כדי למדוד Accuracy או F1 צריך תוויות אמת אמינות.



איור 9 — גרף מומצא של אות לשינוי נתונים, לא של איכות המודל. עלייה באות מחייבת בדיקה, לא מסקנה אוטומטית שהמודל כשל.

במערכת כיתתית אין צורך להקים תשתית ניטור עננית. מספיק להגדיר בדיקה תקופתית: האם טווחי הקלט השתנו? האם מופיעים ערכים חסרים? האם יש דוגמאות חדשות שעליהן המודל נכשל? כל שינוי במודל יחייב גרסה חדשה, תיעוד והערכה הוגנת. אין לשמור נתונים אישיים לצורך ניטור ללא בסיס ואישור מתאימים.

14.19 — פרטיות, רישוי ואחריות

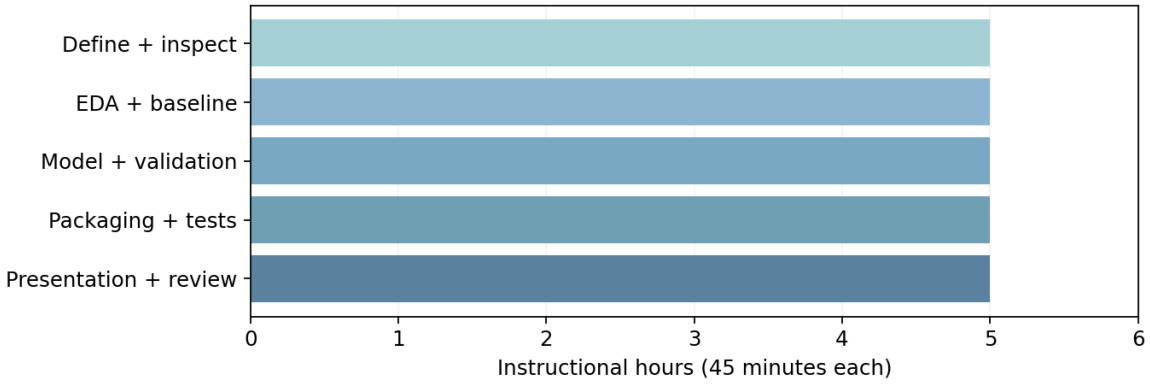
אם הפרויקט משתמש בתמונות של תלמידים, יש לשאול האם בכלל צריך לשמור אותן. אפשר לעבוד תחילה על מאגר ציבורי מתאים ולמנוע איסוף מיותר. כשהפרויקט נוגע לאנשים יש לבדוק ייצוג

אוכלוסיות, נזק אפשרי מטעות, זכויות שימוש ורישוי. אין להבטיח שהמערכת "נטולת הטיה" רק משום שהצליחה על Test אחד.

בפרויקט גמר יש לציין במפורש אם הנתונים מקוריים או פתוחים, מה מותר לעשות בהם, האם נדרש אישור צילום, אילו פרטים נמחקו או הוסוו ואילו מגבלות נשארו. תחזית מודל אינה החלטה מוסמכת בנושאים רפואיים, משפטיים או אחרים בעלי סיכון גבוה.

14.20 — תכנית עבודה מעשית ל־25 שעות

הטבלה שלהלן היא הצעת הוראה מקורית: 25 שעות לימוד לפי חלוקה לחמישה מקטעים בני חמש שעות (שעה לימודית היא 45 דקות). אפשר לפרוס את המקטעים לעשרה מפגשים קצרים יותר. בכל מקטע יש תוצר שמוגש ונבדק.



איור 10 — חלוקת שעות מוצעת, לא חלוקה רשמית של משרד החינוך.

שעות	משימות מובילות	תוצר להגשה
5–1	הגדרת בעיה, כרטיס נתונים, בדיקת שימושיות	Project Brief + Data Card
10–6	EDA, חלוקה, Baseline, הסברים לגרפים	מחברת EDA
15–11	Pipeline, CV, השוואת מודלים, ניתוח טעויות	יומן ניסויים
20–16	שמירת מודל, חוזה קלט, בדיקות, תיעוד	קובץ מודל + בדיקות
25–21	סקירת סיכונים, הצגת תוצר, שיפור והגנה	דוח ופרזנטציה

14.21 — חקר מקרה: האם המודל שבחרנו מתאים למשתמש?

נניח שמודל א׳ משיג Macro F1 של 0.95 אבל זמן חיזוי ממוצע של 400 מילישניות, בעוד מודל ב׳ משיג 0.94 וזמן חיזוי של 8 מילישניות. אין תשובה אוטומטית לשאלה מי טוב יותר. אם המערכת פועלת על טלפון חלש, זמן ההמתנה ומשאבי המכשיר עשויים להיות חשובים. אם טעות סיווג גוררת נזק גדול, הבדל המדד עשוי להיות משמעותי יותר. יש לבדוק שגם דרך המדידה זהה ושאותה קבוצת Test שימשה להשוואה לאחר סיום בחירת המודלים.

המספרים בסיפור זה מומצאים להמחשה. בשאלת הערכה, תלמיד צריך להציע בדיקה שתאפשר לבחור בצורה מקצועית ולא רק לבחור לפי תחושת בטן. אפשר לבקש מדד נוסף, בדיקת קבוצות משתמשים, ניסוי על נתונים חדשים או בדיקת זמן ריצה מדודה.

14.22 — תרגול מדורג בסגנון הערכה

רמה א׳ — הבנה ותיאור

משימה 1: הסבירו במילים שלכם את ההבדל בין תהליך אימון לבין תהליך חיזוי בסביבת משתמש.

משימה 2: ציינו ארבעה פרטים שחייבים להופיע בכרטיס נתונים, והסבירו מה עלול להשתבש כשאחד מהם חסר.

משימה 3: נתון פרויקט שבו כל הקוד נמצא במחברת אחת. הציעו חלוקה לתיקיות והסבירו את התועלת בכל תיקייה.

רמה ב׳ — חישוב, בדיקה והחלטה

משימה 4: מתוך 240 דוגמאות, המודל סיווג 216 נכון. חשבו Accuracy ופירשו את משמעות המדד והמגבלה שלו.

משימה 5: לפרויקט יש 1,000 דוגמאות. חלקו אותן ל־70% אימון, 15% ולידציה ו־15% בדיקה. הסבירו מתי מותר להשתמש בכל חלק.

משימה 6: תלמיד ביצע StandardScaler.fit על כל המאגר לפני החלוקה. הסבירו מדוע זה פוגע באמינות ואיך יש לתקן.

משימה 7: כתבו שלושה מקרי בדיקה לפונקציה שקולטת וקטור בן 64 ערכים. לפחות שניים צריכים להיות מקרי שגיאה.

רמה ג׳ — חקר והגנה על החלטות

משימה 8: בנו תכנית השוואה הוגנת בין KNN ל־Nearest Centroid על digits. פרטו Baseline, ולידציה ומדדי Test.

משימה 9: נסחו תכנית ניטור מצומצמת למערכת המזהה ספרות וכתבו כיצד תבחינו בין Data Drift לבין ירידה מוכחת באיכות.

משימה 10: הציעו שני סיכוני פרטיות או רישוי בפרויקט שמבוסס על תמונות של ילדים, והסבירו כיצד לצמצם אותם.

משימה 11: הגדירו קריטריון הצלחה מדיד לפרויקט אחר: זיהוי פגמים במוצרים או ניבוי זמני משלוח. נמקו בחירת מדד.

משימה 12: כתבו פסקת הגנה: מה הייתם משנים במערכת אם עלתה איכות האימון אבל איכות הבדיקה לא השתפרה?

14.23 — הכנה להסבר בעל־פה ולפרויקט גמר

בעת הגנה על פרויקט אין די במשפט "הרצתי מודל וקיבלתי 96%". יש לתאר את שאלת המחקר, מקור הנתונים, מה הוזן למודל, כיצד נבחרו הפרמטרים, אילו מדדים נמדדו, כיצד נמנעה זליגת מידע ומהן המגבלות. על התלמיד להיות מסוגל להדגים שינוי קטן בקוד, להציג בדיקה ולהסביר את תוצאתה.

דוגמאות לשאלות בוחן: מדוע בחרתם דווקא בנתונים האלה? כיצד אתם יודעים שמודלכם לא ראה את Test בזמן האימון? האם תוכלו לשחזר ניסוי ישן? מה קורה אם המשתמש שולח תמונה בפורמט אחר? האם שמירת מודל מבטיחה שהוא יפעל במחשב אחר? מה תעשו אם התפלגות הנתונים תשתנה?

מחוון כיתתי מוצע לפרק — שאינו מחוון משרד החינוך: הגדרת בעיה ונתונים 20 נקודות; תהליך ניסוי מבוקר 25; בדיקות וקוד מסודר 20; תיעוד ושחזור 15; ניתוח מגבלות ופרטיות 10; הצגה והגנה בעל־פה 10.

14.24 — מפת ההקשר בין שלוש תכניות הלימוד

תפקיד בהמשך	ידע שנדרש או נלמד	מסלול
בסיס לניתוח ולהסבר מודל	Pandas, גרפים, KNN ומדדי הצלחה	יחידה 3 — כיתה י׳
ביסוס מתמטי והערכה מדעית	מרחקים, K-Means, הערכה וולידציה	יחידה 5 — נושאי ליבה
הכנה ישירה לתוצר תוכנה ולעבודת גמר	ארגון קוד ונתונים, שמירת מודל, בדיקות ויישום	למידה עמוקה — התמחות

הקשר בין המסלולים הוא מיפוי דידקטי שלנו. כל דרישה פורמלית נבחנת בהתאם לתכנית ולמחוון הרלוונטיים, ואין להעתיק אוטומטית עומק של מסלול אחד לאחר.

14.25 — סיכום הפרק: מה הופך ניסוי לפרויקט מקצועי?

בשלב זה התלמיד כבר יודע לא רק לאמן מודל, אלא לחשוב כמו מפתח AI: להגדיר בעיה, לבחור נתונים, לתעד ניסויים, למנוע זליגה, לשמר את תהליך העיבוד, להגדיר חוזה קלט ולבדוק מקרים צפויים וחריגים. פרויקט איכותי אינו מדד יחיד ואינו רק Notebook ארוך; הוא רצף החלטות שאפשר לשחזר ולנמק.

בפרק 15 נלמד רשתות חוזרות (RNN). בהמשך, בפרק 16, נשתמש בכל תהליכי הניהול, התיעוד והבדיקה שנלמדו כאן לצורך עבודת גמר המבוססת על מודל למידה עמוקה ועל יישום תוכנה עצמאי.

נספח א׳ — דף תכנון פרויקט למילוי

זהו טופס עבודה לתלמיד. אין צורך למלא את כל הסעיפים לפני המפגש הראשון: יש לחזור אליו לאחר כל ניסוי ולתקן החלטות לפי הראיות. בכל שדה יש לכתוב תשובה קונקרטית שניתן לבדוק ולא סיסמה כללית.

מה נכתוב בפרויקט שלנו	השאלה שעלינו לתעד
_____	איזו בעיה נפתור, עבור מי ולמה?
_____	מה מקור הנתונים ומה תנאי השימוש?
_____	מהו מאפיין ומהי תווית המטרה?
_____	כיצד נחלק ל־Train, Validation ו־Test?
_____	מהו קו הבסיס ומה המדד הראשי?
_____	איזה ניסוי נבצע ראשון ומדוע?

מה נכתוב בפרויקט שלנו	השאלה שעלינו לתעד
_____	כיצד נבדוק קלט חדש למודל?
_____	מהם סיכון אחד ומגבלה אחת?

נספח ב׳ — יומן ניסויים למילוי

רשמו כל הרצה כראיה הניתנת לשחזור. אין למחוק תוצאה חלשה: גם כישלון מתועד עוזר לבחור ניסוי המשך. הקפידו לא לערבב ציוני Train, CV ו־Test באותה עמודה ללא ציון סוג המדידה.

מזהה ניסוי	מה שיניתי?	מדד + סוג בדיקה	מה הסקתי?
E01	_____	_____	_____
E02	_____	_____	_____
E03	_____	_____	_____
E04	_____	_____	_____

נספח ג׳ — רשימת בדיקה לפני הדגמה לבוחן

לפני ההגשה, הריצו את המחברת מההתחלה בסביבה נקייה וודאו שכל התאים החיוניים עובדים לפי הסדר. פתחו README במחשב אחר או בסביבה חדשה ובדקו שאפשר להבין כיצד מתקינים ומפעילים. הציגו דוגמה שהמודל מסווג נכון ודוגמה שנכשל בה, עם ניתוח ולא עם הסתרה. בדקו שניתן לשחזר חיזוי ממודל שמור ושבדיקת קלט לא חוקי מחזירה שגיאה מובנת.

מומלץ לשמור בפרויקט תמונת מסך של גרף בקרה, קובץ מדדים ברור, פירוט גרסאות ספריות, כרטיס נתונים וכרטיס מודל. אם הקובץ מכיל מידע רגיש יש להסירו לפני שיתוף. על כל תלמיד להיות מסוגל להצביע על קוד שחיבר בעצמו ולהסביר את ההיגיון שלו גם ללא עזרת כלי AI.

נספח ד׳ — כרטיס מודל קצר להגשה

כרטיס מודל הוא דף תיעוד שמסביר איזה מודל נבנה, לאיזו מטרה, על אילו נתונים, כיצד הוערך ומתי אסור להסתמך עליו. זהו כלי תיעוד מקצועי שמאפשר למישהו אחר להבין את ההחלטות שלכם, ולא קובץ שנוצר אוטומטית על ידי התוכנה.

שדה בכרטיס המודל	דוגמה שמראה מה נדרש
שם וגרסה	DigitClassifier, version 1.0
מטרת המודל	סיווג ספרה בודדת בתמונה 8×8
מקור נתונים	sklearn digits + תנאי שימוש
קלט / פלט	64 ערכי פיקסל 0–16 / ספרה 0–9
פרוטוקול בדיקה	Train 80% + CV פנימי, Test חיצוני 20%

דוגמה שמראה מה נדרש	שדה בכרטיס המודל
Accuracy, Macro F1 ומטריצת בלבול	מדדי איכות
לא נבדק על תמונות טלפון ברזולוציה שונה	מגבלות חשובות
גרסת Python ו־scikit-learn, קובץ Pipeline מהימן	גרסאות וקבצים

בוחן יכול לשאול: האם זו מערכת לזיהוי ספרות באופן כללי? התשובה המדויקת היא שהיא נבדקה על נתונים הדומים ל־digits; טענה רחבה יותר דורשת בדיקה עצמאית על אוכלוסיית תמונות חדשה. זהו ההבדל בין טענה מגובה לבין הבטחה שאיננה מבוססת ניסוי.

מקורות לקריאה ולהעמקה

עוגן מסלול: מאגר הקורס — פרק 14, 25 שעות (מפת התכנית; דורש הצלבה מול הסילבוס הרשמי המלא): https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/14-Data-Science-Project-Management

ארגון ניסויים, אימון והערכה. — Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — James et al., An Introduction to Statistical Learning with Applications in Python — Zhang et al., Dive into Deep Learning. ניסויים ויישום. דוגמאות בחירת מודלים והערכת ביצועים. והסברים בפרק זה מקוריים.

scikit-learn — Common Pitfalls and Recommended Practices: https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn — Model Persistence: https://scikit-learn.org/stable/model_persistence.html

Google for Developers — Rules of Machine Learning: https://developers.google.com/machine-learning/guides/rules-of-ml

Google for Developers — Productionization: https://developers.google.com/machine-learning/managing-ml-projects/production

DVC — User Guide (העשרה בלבד): https://doc.dvc.org/user-guide

הערת מקור: הספר אינו תרגום של ספר מסחרי. ההסברים, הדוגמאות, השאלות והתרשימים נוצרו לצורכי הוראה. אין לייחס לכלי כגון DVC, Git או joblib מעמד של דרישת חובה רשמית ללא אימות מול התכנית.